Module 9 · SW Design

Agentic Design & Coding

How to design and build software with an AI agent — as its director, not its passenger.

Akın Kaldıroğlu · Lead Trainer

Where we are

From Writing Code to Directing It

You now have the design vocabulary. This session is where it pays off: it is exactly what lets you steer an AI toward good software.

The agent

Plans, edits files, runs tools and tests in a loop.

You

Frame the problem, judge the design, own the decisions.

The bar

Everything from Phase 1: cohesion, coupling, SOLID, tests.

What you'll leave with

Learning Objectives

- Understand what an AI coding agent is — the loop, and the harness around it.
- Drive a design conversation: intent → plan → build → review.
- Engineer the context, not just the prompt — select, persist, retrieve, compact.
- Keep quality high — you are the reviewer and decision-maker.

The idea

What Makes It 'Agentic'

A chatbot answers. An agent acts: it reads your code, makes a plan, edits files, runs tests, sees the results, and corrects itself — in a loop.

A plain assistant

- You copy-paste snippets back and forth
- No access to your project
- One turn at a time

An agent (e.g. Claude Code)

- Reads and edits your real files
- Runs commands, tests, and tools
- Loops: act → observe → adjust

The idea

The Harness Around the Model

A model produces text. What turns it into an agent is the harness around it — the loop, the tools, the limits and the checks. Same model, different harness, different results.

The loop

Act → observe → correct, repeated until the goal is met or a limit stops it. Nothing else in the harness matters without this.

Tools

Read and write files, run commands and tests, search, call services. What the harness does not expose, the agent simply cannot do.

Permissions & scope

What it may touch unasked, what needs your approval, where it must not go. Guardrails are part of the design, not an afterthought.

Verification

Tests, types, linters, a build that runs. This is how the loop tells a working change from a merely plausible one.

Claude Code is a harness — and so are your project instructions, your test suite and your CI. You are building one whether you mean to or not.

Section 01

The Workflow

A repeatable loop: intent, plan, build, review.

Workflow

Intent → Plan → Build → Review

1 · Intent — state the goal, constraints & context

2 · Plan — agree the design before any code

3 · Build — let the agent implement in small steps

4 · Review — judge, test, and correct

The loop repeats in small cycles — not one giant prompt.

Workflow

Prompt for Design, Not Just Code

Vague prompts get vague code. Give the agent the same brief you'd give a junior engineer: the goal, the constraints, and the shape you want.

Weak prompt

- 'Write an order service'
- No constraints, no context
- You'll get someone's default

Strong prompt

- State the goal and the rules
- Name the design: 'depend on a Repository port'
- Point at the files and patterns to follow

Workflow

Same Problem, Two Prompts

One business rule, handed to the agent twice — the second time with the design named:

Prompt 1

“Calculate the shipping fee: standard 5, express 15 + 1/kg, overnight 30 + 2/kg.”

what you get

```
double fee(Order o) {
  if (o.method() == STANDARD)  return 5;
  if (o.method() == EXPRESS)   return 15 + o.kg();
  if (o.method() == OVERNIGHT) return 30 + o.kg()*2;
  throw new IllegalArgumentException(o.method());
}

// a new shipping method?
// edit this method again
```

Prompt 2

“…the same, and implement it with the Strategy pattern.”

what you get

```
interface ShippingFee { double of(Order o); }

class Standard implements ShippingFee {
  public double of(Order o){ return 5; } }
class Express implements ShippingFee {
  public double of(Order o){ return 15 + o.kg(); } }
class Overnight implements ShippingFee {
  public double of(Order o){ return 30 + o.kg()*2; } }
// a new method = a new class; nothing above changes
```

Same behavior, same tests — one word of design in the prompt. The agent had no opinion; you did.

Which is why Phase 1 matters here: you can only ask for the design you can name.

Workflow

Conversation Techniques

- Work in small steps — review each before moving on.
- Ask for 2–3 approaches with trade-offs before committing.
- Make it explain its design; if it can't, that's a red flag.
- Correct course early — don't let a bad direction compound.

Workflow

Context Engineering

Prompting is one line of it. Context engineering is deciding everything the model can see when it answers — and, just as deliberately, what it must not.

Give it

- The files that matter for this task
- Project conventions & standards
- Examples of the style you want

Spare it

- The entire repository at once
- Irrelevant history and noise
- Ambiguity about what 'done' means

Workflow

Five Moves for Context

The window is finite and everything in it competes for attention. These are the moves you actually make:

Select

The files and conventions this task needs — not the whole repository. Precision beats volume.

Persist

Put standing rules in project instructions (CLAUDE.md, AGENTS.md) so you never retype them.

Retrieve

Let it search and read the code itself. Freshly read beats pasted and stale.

Compact

Summarize, prune, or start a clean thread when the conversation has wandered.

Isolate

Give a self-contained sub-task its own sub-agent and context, so the main thread stays clean.

Poor output is usually a context problem before it is a model problem.

Section 02

Staying in Control

The agent is fast. You are responsible.

Control

Review With Your Design Vocabulary

This is why Phase 1 matters. You review AI output through the same lens you'd use on a colleague's pull request.

- Cohesion & coupling: is each part focused and independent?
- SOLID: any SRP or DIP violations sneaking in?
- Clean code: names, size, honest errors, no duplication?
- Security & tests: validated inputs, covered edge cases?

Control

How a Pull Request Works

The agent's changes reach you as a pull request — a proposal to merge a branch, reviewed before it lands. Nothing changes in main until you approve.

1 · Branch — the agent works on a branch, never straight on main

2 · Open PR — it proposes: "merge this branch into main"

3 · Review the diff — read every change; comment, request changes, approve

4 · Merge — it lands only when you click merge; you are the gate

Same flow whether the change comes from a teammate or your AI agent — you review, you decide.

Control

Guardrails Beat Vigilance

Let the machine check

- Tests — run them every loop
- Type checks, linters, formatters
- Security scanners on the diff

You focus on

- Is this the right design?
- Does it meet the real requirement?
- The costly, architectural decisions

Control

Anti-Patterns to Avoid

Blind acceptance

Merging code you don't understand. If you can't review it, don't ship it.

One mega-prompt

Asking for everything at once. Work in small, reviewable steps.

Losing the thread

Letting the agent drift. You hold the goal and the standards.

Over to you

Can You Outsource the Thinking?

> *“you can outsource your thinking*
>
> *but you cannot outsource your understanding”*
>
> *— @yacineMTB (kache), cited by Andrej Karpathy*

But can the two be separated at all?

If the agent did the thinking, where did your understanding come from?

→ **What do you think?**

the original post → `x.com/karpathy/status/2049907410303865030`

Remember these

Key Takeaways

- ✓ An agent acts in a loop: plan, edit, run, observe, correct.
- ✓ Work the loop: intent → plan → build → review, in small steps.
- ✓ Prompt for design; engineer the context; ask for alternatives.
- ✓ Review AI output with cohesion, coupling, SOLID, clean-code and security.
- ✓ The agent is fast; you are the decision-maker and the reviewer.

Go deeper

Further Reading

This chapter moves faster than publishing does — books for the method, primary sources for what changed this month.

📖 **Beyond Vibe Coding: From Coder to AI-Era Developer** Addy Osmani · O'Reilly, 2025

The closest match to this session: intent-driven workflow, and staying the director of the agent.

📖 **AI-Assisted Programming: Better Planning, Coding, Testing, and Deployment** Tom Taulli · O'Reilly, 2024

AI across the whole cycle — planning, coding, testing, docs — with a modular method that suits prompting.

Anthropic Engineering — "Building Effective Agents" → `anthropic.com/engineering`

Claude Code & Agent SDK — the official documentation → `docs.claude.com`

Next session

What's Next

Module 10 · Saturday 1 August

SW Design Project with AI

with Akın Kaldıroğlu

You'll build a real feature end-to-end with Claude Code — applying every principle and technique from Phase 1.

Further practice

Drive a Refactoring with AI

Take the ACME Orders capstone and refactor it end-to-end with an agent — you decide each move, the AI executes it.

Refactoring course · Capstone

Repo `github.com/kaldiroglu/refactoring-course-student-materials`

IN THIS SESSION, LOOK AT

- `Exercises/Capstone-Brief.md`
- `Projects/RefactoringCapstone-Java, -CSharp, -Python`

Questions?

Direct the agent. Own the design.

Akın Kaldıroğlu · Agentic Design & Coding